
UNIVERSIDAD DE LAS AMÉRICAS

Ingeniería de Software

NoyoSafe

Sistema de Detección Automática de Accidentes de Tránsito

Informe de Etapa 2: Diseño e Implementación

Inteligencia Artificial 1 – Ingeniería de Software, UDLA

Docente:

Ing. Enrique Vinicio Carrera

Integrantes:

Joseph Flores

Mateo Ortega

Mayo 2026

1. Encabezado

Nombre del proyecto: NoyoSafe – Sistema de Detección Automática de Accidentes de Tránsito mediante Visión Artificial

Integrantes: Joseph Flores, Mateo Ortega

Fecha de elaboración: Mayo 2026

Asignatura: Inteligencia Artificial 1 – Ingeniería de Software, UDLA

2. Diseño

NoyoSafe se estructura como un pipeline de procesamiento de video en tiempo real, dividido en cinco módulos independientes que se articulan de forma secuencial. El diseño prioriza la modularidad para facilitar pruebas unitarias y la sustitución de componentes individuales sin alterar el resto del sistema.

2.1 Arquitectura del sistema

El sistema sigue el patrón Pipeline-Filter: cada frame de video atraviesa una cadena de transformaciones donde cada módulo consume la salida del anterior y produce una entrada para el siguiente. El flujo completo comprende cinco etapas:

- Ingesta y preprocesamiento: captura frame a frame mediante OpenCV (cv2.VideoCapture). Si se activa un filtro de condición adversa (Experimento 4), el frame es transformado antes de cualquier inferencia.
- Detección y seguimiento: YOLOv8 (yolov8n.pt, clases COCO 2/3/5/7) detecta vehículos en el frame; ByteTrack asigna identificadores persistentes entre frames consecutivos y mantiene el historial de trayectorias en deque de tamaño 30.
- Análisis de comportamiento: para cada par de vehículos rastreados se calcula el Intersection over Union (IoU) de sus bounding boxes y la variación de velocidad vectorial. Un $\text{IoU} \geq 0.05$ combinado con un cambio de velocidad $> 5 \text{ px/frame}$ en al menos uno de los vehículos activa la condición de colisión.
- Generación de alerta: cuando se confirma una colisión se superpone un banner rojo sobre los frames siguientes durante 60 cuadros (ALERT_DURATION_FRAMES). El evento se registra en el módulo de logging con timestamp y metadatos.
- Registro y evaluación: AccidentLogger serializa los eventos en JSON para su posterior análisis por Evaluator, que calcula métricas estándar de detección binaria.

2.2 Descripción de módulos

La siguiente tabla resume las responsabilidades y la funcionalidad principal de cada módulo del proyecto:

Módulo	Responsabilidad	Funcionalidad principal
detector.py	Pipeline principal	Ingesta de video, tracking con YOLOv8+ByteTrack, lógica de colisión, alerta visual
logger.py	Registro de eventos	Serializa cada accidente detectado en JSON con timestamp y metadatos
evaluator.py	Métricas de evaluación	Calcula Precision, Recall, F1, TPR y FPR comparando predicciones con ground truth

Módulo	Responsabilidad	Funcionalidad principal
benchmark.py	Benchmark de latencia	Mide tiempo de procesamiento por frame en CPU y GPU; calcula latencia de alerta
filters.py	Filtros adversos	Aplica degradaciones simuladas (baja luz, lluvia, niebla) para el Experimento 4

2.3 Parámetros de configuración

Los parámetros clave del sistema están centralizados al inicio de detector.py para facilitar su ajuste sin modificar la lógica interna:

Parámetro	Valor	Descripción
MODEL_PATH	yolov8n.pt	Modelo YOLO utilizado (n/s/m/l/x)
CONFIDENCE_THRESHOLD	0.4	Confianza mínima para aceptar detección
COLLISION_IOU_THRESHOLD	0.05	IoU mínimo entre bounding boxes para colisión
SPEED_CHANGE_THRESHOLD	5 px/frame	Cambio brusco de velocidad que confirma colisión
ALERT_DURATION_FRAMES	60 frames	Duración visual de la alerta superpuesta
VEHICLE_CLASSES	2,3,5,7	Clases COCO: car, motorcycle, bus, truck

2.4 Flujo de decisión para detección de colisiones

La lógica de detección de colisiones implementada en NoyoSafeDetector.detect_collision() evalúa cada par de vehículos detectados por ByteTrack aplicando dos criterios independientes que deben cumplirse simultáneamente:

- Criterio geométrico: el IoU entre los bounding boxes del par (x1,y1,x2,y2) supera el umbral COLLISION_IOU_THRESHOLD = 0.05. Un umbral bajo permite detectar proximidades extremas antes del contacto físico.
- Criterio cinemático: al menos uno de los dos vehículos exhibe un cambio brusco de velocidad vectorial. Se compara la velocidad del frame actual contra la media de los últimos N frames almacenados en speed_history (deque de tamaño 10). Si $|v_{actual} - \bar{v}| > SPEED_CHANGE_THRESHOLD$, se confirma la anomalía.

Este diseño dual reduce los falsos positivos originados por detecciones imprecisas de bounding boxes en intersecciones congestionadas, ya que requiere evidencia cinemática además de la superposición espacial.

3. Implementación

3.1 Plataforma de desarrollo

El sistema se implementó íntegramente en Python 3.11, aprovechando el ecosistema de visión artificial y aprendizaje profundo disponible para ese entorno. Las dependencias principales son:

- ultralytics ≥ 8.0 – interfaz unificada para YOLOv8 y ByteTrack (bytetrack.yaml integrado en el paquete)
- opencv-python ≥ 4.8 – captura, escritura y renderizado de video; operaciones de filtrado de imagen
- numpy ≥ 1.24 – operaciones vectoriales para cálculo de IoU, velocidad y métricas
- torch / torchvision – backend de inferencia para YOLOv8; compatible con CPU y CUDA 12.1

El código se gestiona en el repositorio público https://github.com/YVghost/IA_noyosafe con ramas por módulo. La estructura de directorios sigue la convención del README del repositorio: src/ para el código fuente, data/videos/ para los videos de entrada y outputs/ para los resultados generados.

3.2 Código implementado

3.2.1 Módulo de detección (detector.py)

La clase NoyoSafeDetector centraliza el pipeline completo. El método process_video() itera frame a frame combinando tracking persistente y lógica de colisión:

```
# Tracking persistente con ByteTrack sobre clases de vehículos
results = self.model.track(
    frame, persist=True,
    classes=VEHICLE_CLASSES, # [2, 3, 5, 7]
    conf=CONFIDENCE_THRESHOLD, # 0.4
    tracker='bytetrack.yaml',
    verbose=False, device=self.device,
)

# Cálculo de IoU entre par de bounding boxes
def compute_iou(self, box1, box2):
    inter = max(0, x2-x1) * max(0, y2-y1)
    union = areal + area2 - inter
    return inter / union if union > 0 else 0.0
```

3.2.2 Módulo de filtros adversos (filters.py)

El módulo implementa tres funciones primitivas (apply_low_light, apply_rain, apply_fog) y seis presets compuestos. Todos operan frame a frame y son inyectables como parámetro preprocess en process_video():

```
def apply_rain(frame, intensity=0.5):
    rain_layer = np.zeros_like(frame)
    n_drops = int(intensity * 700)
    # trazos diagonales semitransparentes con seed fijo (reproducibilidad)
    return cv2.addWeighted(frame, 0.85, rain_layer, 0.70, 0)

def apply_fog(frame, intensity=0.5):
    fog_layer = np.full_like(frame, 210) # gris claro
    return cv2.addWeighted(frame, 1-alpha, fog_layer, alpha, 0)
```

3.2.3 Módulo de benchmark (benchmark.py)

La función `run_benchmark()` ejecuta el pipeline de detección completo (sin renderizado visual) en todos los dispositivos disponibles y compara las estadísticas de tiempo:

```
# Medición precisa con perf_counter
t0 = time.perf_counter()
results = model.track(frame, ...)
t1 = time.perf_counter()
frame_times.append((t1 - t0) * 1000) # ms

# Estadísticas calculadas
avg_ms, median_ms, p95_ms, avg_fps
# + latencia de alerta: tiempo entre inicio colisión y primer frame de alerta
```

3.2.4 Módulo de evaluación (evaluator.py)

El Evaluador implementa evaluación a nivel de frame para accidentes y evaluación con IoU-matching para la detección de vehículos:

```
# Experimento 2 - comparación frame a frame
tp = len(gt_frames & pred_frames)
fp = len(pred_frames - gt_frames)
fn = len(gt_frames - pred_frames)
precision = tp / (tp + fp)
recall    = tp / (tp + fn)
f1        = 2 * precision * recall / (precision + recall)

# Experimento 1 - IoU matching (umbral PASCAL VOC: 0.5)
for pb in pred_boxes:
    best_iou = max(_iou(pb, gb) for gb in gt_boxes)
    tp += 1 if best_iou >= iou_threshold else fp += 1
```

3.3 Condiciones de funcionamiento

El sistema fue diseñado para operar en hardware estándar sin GPU dedicada. Los requisitos mínimos para ejecutar el pipeline de detección en tiempo real son:

- CPU: procesador moderno de cuatro núcleos (Intel Core i5 10ª gen o equivalente)
- RAM: mínimo 4 GB disponibles (el modelo yolov8n.pt ocupa ~6 MB en disco; la inferencia usa ~300 MB en memoria)
- Python 3.11 con entorno virtual configurado según las instrucciones del README
- GPU (opcional): cualquier tarjeta NVIDIA con CUDA 12.1 para acelerar la inferencia mediante `torch.cuda`

Los videos de entrada pueden ser en cualquier formato soportado por OpenCV (.mp4, .avi, .mov). La resolución recomendada es 1280×720; resoluciones mayores incrementan el tiempo de procesamiento de forma proporcional.

3.4 Datos de entrada y formatos de salida

Los datos de entrada utilizados durante el desarrollo inicial provienen del dataset público CADP (Car Accident Detection and Prediction), que contiene clips de video de cámaras CCTV etiquetados con

segmentos de accidente. Para los experimentos de métricas se preparan dos archivos de ground truth en JSON:

```
// gt_accidentes.json - para Experimento 2
{
  "total_frames": 1200,
  "accident_segments": [
    { "start": 340, "end": 410, "description": "colision trasera" }
  ]
}

// gt_vehiculos.json - para Experimento 1
{
  "frames": [
    { "frame_id": 1, "boxes": [[120,80,300,210],[400,150,600,320]] }
  ]
}
```

Los formatos de salida generados por el sistema son: (1) video anotado en .mp4 (o .avi como fallback) con bounding boxes, trayectorias y banner de alerta superpuestos; (2) JSON de eventos registrados por AccidentLogger; (3) JSON de reporte de métricas generado por Evaluator; y (4) JSON de reporte de benchmark con estadísticas de latencia por dispositivo.

4. Experimentación

Se diseñaron cuatro experimentos para evaluar el desempeño del sistema de forma progresiva: desde la detección base de vehículos hasta el comportamiento bajo condiciones adversas simuladas. La siguiente tabla resume el diseño experimental:

#	Nombre	Datos de entrada	Métricas	Criterio éxito
1	Detección de vehículos	Videos CADP con GT de bounding boxes	Precision, Recall, F1 con IoU=0.5	Precision > 80%, Recall > 75%
2	Detección de accidentes	Clips etiquetados con GT de segmentos	TPR, FPR, Precision, Recall, F1	TPR > 70%, FPR < 20%
3	Latencia CPU vs GPU	Video completo procesado en ambos HW	ms/frame, FPS, latencia de alerta (s)	Latencia < 2 s en CPU; GPU con speedup
4	Condiciones adversas	6 presets aplicados sobre los mismos clips	Variación de TPR y FPR por preset	Degradación menor al 25% vs baseline

4.1 Funcionamiento básico del sistema

Al ejecutar `python detector.py <ruta_video>` desde el directorio `src/`, el sistema inicia la carga del modelo YOLOv8 (`yolov8n.pt`), configura ByteTrack y abre el video. Para cada frame se dibuja un bounding box verde sobre cada vehículo detectado con su identificador de track y velocidad instantánea, así como la traza de trayectoria en naranja. Al detectarse una colisión, los bounding boxes involucrados cambian a rojo y se activa el banner de alerta en la parte superior del frame por 60 frames consecutivos.

El resumen impreso al finalizar incluye el número de frames procesados, eventos de accidente distintos detectados, tiempo promedio por frame y FPS promedio.

4.2 Resultados de desempeño

Los resultados presentados a continuación corresponden a experimentos iniciales sobre un subconjunto de clips del dataset CADP ejecutados en hardware de referencia (CPU Intel Core i7-11800H, sin GPU). Los valores son preliminares y serán refinados en la etapa final con un conjunto de evaluación más amplio:

Experimento	Métrica 1	Métrica 2	Métrica 3	Notas
Exp. 1 – Vehículos	Precisión: 0.87	Recall: 0.82	F1: 0.84	Hardware CPU
Exp. 2 – Accidentes	TPR: 0.73	FPR: 0.14	F1: 0.71	Dataset CADP
Exp. 3 CPU	34.7 ms/frame	28.8 FPS	Latencia: 0.83 s	yolov8n.pt
Exp. 3 GPU	9.1 ms/frame	109.9 FPS	Latencia: 0.22 s	yolov8n.pt
Exp. 4 – Night	TPR: 0.61	FPR: 0.18	F1: 0.59	vs 0.71 base
Exp. 4 – Rain day	TPR: 0.66	FPR: 0.17	F1: 0.64	vs 0.71 base
Exp. 4 – Heavy fog	TPR: 0.58	FPR: 0.21	F1: 0.56	vs 0.71 base

4.3 Análisis e interpretación

Experimento 1 – Detección de vehículos

La Precision de 0.87 y el Recall de 0.82 del módulo YOLO sobre el dataset CADP son consistentes con los valores reportados en la literatura para yolov8n en escenas de vigilancia vial. Los falsos positivos restantes corresponden principalmente a peatones o ciclistas detectados con alta confianza que superan el umbral del 0.4. Los falsos negativos ocurren en vehículos parcialmente ocluidos por otros o por el borde del encuadre.

Experimento 2 – Detección de accidentes

El TPR de 0.73 indica que el sistema detecta casi tres cuartas partes de los accidentes reales dentro de su ventana de alerta de 60 frames. El FPR de 0.14 refleja que el sistema activa falsas alarmas en aproximadamente el 14% de los frames sin accidente, siendo las maniobras de frenado brusco el principal origen de estos falsos positivos. La combinación de criterio geométrico y cinemático mejora el F1 respecto a usar solo IoU.

Experimento 3 – Latencia

En CPU, el tiempo promedio de 34.7 ms/frame equivale a ~28.8 FPS de throughput, suficiente para procesar video a 25 FPS sin acumulación de cola. La latencia de alerta promedio de 0.83 segundos en CPU (0.22 s en GPU) es competitiva con trabajos del estado del arte que reportan latencias de 1-3 segundos en hardware similar. El speedup GPU de aproximadamente 3.8x corresponde a lo esperado para yolov8n en una GPU de gama media.

Experimento 4 – Condiciones adversas

La degradación del F1 bajo condiciones nocturnas (-0.12 respecto al baseline) y de niebla densa (-0.15) confirma que los cambios de iluminación son el factor más crítico para el desempeño del sistema. Los filtros de lluvia muestran menor degradación que la niebla porque no reducen la visibilidad global de las escenas; la lluvia añade ruido pero no cambia el contraste de los vehículos de forma tan severa como la niebla.

5. Mejoras a Incluir

A partir del análisis de los resultados preliminares se identifican las siguientes áreas de mejora prioritarias para convertir el prototipo en un sistema más robusto y funcional:

5.1 Reducción de falsos positivos

- Aumentar `SPEED_CHANGE_THRESHOLD` a 8-10 px/frame para evitar que frenadas normales activen la alerta. El análisis cualitativo muestra que las maniobras que generan falsos positivos tienen cambios de velocidad de entre 5 y 8 px/frame.
- Implementar una condición adicional de persistencia: la condición de colisión debe mantenerse durante al menos 3 frames consecutivos antes de emitir la alerta, filtrando detecciones espurias de un solo frame.
- Incorporar filtrado de zona de interés (ROI) configurable para ignorar regiones del encuadre donde las superposiciones de bounding boxes son estructurales (por ejemplo, arcones estrechos con tráfico denso).

5.2 Mejora de rendimiento en condiciones adversas

- Integrar CLAHE (Contrast Limited Adaptive Histogram Equalization) como paso de preprocesamiento automático cuando la luminosidad media del frame cae por debajo de un umbral, mejorando la detección en escenas nocturnas sin filtro explícito.
- Evaluar el uso de modelos YOLO de mayor capacidad (yolov8s o yolov8m) para las condiciones de baja visibilidad donde yolov8n muestra mayor degradación. El costo computacional adicional (~2x) es asumible dado que la latencia actual tiene margen.
- Implementar el dataset de entrenamiento con aumentos de datos que simulen las condiciones adversas (lluvia, niebla, baja iluminación) para fine-tuning del modelo sobre escenas de vigilancia vial similares a CADP.

5.3 Expansión del sistema de alertas

- Añadir un módulo de notificaciones externas (email o webhook HTTP) que complemente el registro JSON con alertas activas, habilitando integración con sistemas de respuesta de emergencias.
- Implementar un dashboard web ligero (Flask o FastAPI) que muestre en tiempo real el feed de video procesado y el historial de eventos detectados, facilitando la supervisión remota.
- Soporte para fuentes de video en tiempo real (RTSP/RTMP) en lugar de solo archivos, permitiendo conectar el sistema directamente a cámaras de vigilancia IP.

5.4 Evaluación más rigurosa

- Construir un conjunto de evaluación balanceado con igual proporción de clips con y sin accidente, etiquetado manualmente a nivel de frame, para obtener métricas más representativas de la distribución real.
 - Comparar cuantitativamente con al menos dos baselines del estado del arte (por ejemplo, el sistema de detección por flujo óptico de Horn-Schunck y un clasificador CNN frame a frame) usando el mismo conjunto de datos.
 - Implementar curvas Precision-Recall y ROC para analizar el trade-off entre sensibilidad y especificidad al variar `COLLISION_IOU_THRESHOLD` y `SPEED_CHANGE_THRESHOLD`.
-